Formulas · delays · switching · multiplexing · TCP · IPv4

Units

- Bytes × 8 = bits · 1 KB = 1,000 B · 1 Mb/s = 10^6 bits/s
- 1 ms = 10^{-3} s · 1 µs = 10^{-6} s · km/s × 1,000 = m/s

Single-link delays

- **Transmission** T_t = L (bits) ÷ R (bits/s) *: 160 B at 20 Mb/s = 64 µs*
- **Propagation** T_p = d ÷ s *: T_p doesn't depend on L or R; T_t doesn't depend on d*
- **Packet delay** = T_t + T_p (+ queueing) *: 180 B, 10 Mb/s, 15 µs → 159 µs*
- **RTT** = 2 × (T_t + T_p) *: 250 B, 125 Mb/s, 73.75 km, 295M m/s → 532 µs*
- **Bandwidth-delay product** = R × T_p (bits) *: 80 Mb/s × 4 µs = 320 bits*
- **Packets on the link** (also asked as "packets stored on the link" or "transmitted and propagating before the first bit arrives") = ⌊T_p ÷ T_t⌋ *: 60 km, 2 Gb/s, 800 B → 62*
- **Upload time** = file (bits) ÷ R *: 2,000 KB at 4 Mb/s = 4 s*
- **Clock cycles** = time × clock rate *: 25 m and back = 166.8 ns × 5 GHz → 834*
- **Packet error prob** ≈ bits × p *: 1,500 B × 10^{-6} ≈ 1%*
- **Frame error** p = 1 − $(1 - p_b)^L$ (exact; ≈ L × p_b only if p ≪ 1) · max frame L < ln(1 − p) ÷ ln(1 − p_b) *: 5%, 40 in a million → 160 B*
- **Frames sharing one check sequence** = $2^{\text{data bits}}$ ÷ $2^{\text{check bits}}$ *: 5 B data, 16-bit check → 2^{24}*

Store-and-forward paths

- **Hops** h = intermediate nodes k + 1
- **One packet, same rate:** h × (T_t + T_p) *: 6 nodes, 1,500 B, 150 Mb/s → 7 × 80 = 560 µs*
- **Different link rates:** Σ (L ÷ R_i) + Σ T_p *: 200 B over 10 and 20 Mb/s → 160 + 80 = 240 µs*
- **Max packet for delay D:** L = D × R ÷ h, ÷ 8, floor *: 350 µs, 150 Mb/s, 7 hops → 937 B*
- **N packets, pipelined:** (N + k) × T_t + h × T_p
- **Circuit:** setup + M ÷ R + h × T_p (+ release)
- **Packet switching same or faster than circuit** when k × T_t ≤ setup, for any message size (propagation and M ÷ R cancel). **Largest packet** = setup × R ÷ (8k) bytes *: 5 nodes, 1,500 B, 10 Mb/s → 6 ms · 7 nodes, 84 Mb/s, 30 ms → 45 KB*

A —L1— B —L2— C (two packets)

- **No queueing at B** ⇔ T(P2 on L1) ≥ T(P1 on L2) ⇔ **P2 ÷ R1 ≥ P1 ÷ R2** *(propagation cancels)*
- Min P2 = P1 × R1 ÷ R2 · Max P1 = P2 × R2 ÷ R1 · **check the units asked for**: bits = bytes × 8 *: P1 = 1,200 B = 9,600 bits, 25 / 200 Mb/s → min P2 = 9,600 × 25 ÷ 200 = 1,200 bits : P2 1,200 B, 160 / 100 Mb/s → 750 B*
- **Time C gets P2:** P2 starts on L2 at max(P2 fully at B, B done with P1), then + T(P2 on L2) + T_{p2} *: 387 µs*

Statistical multiplexing

- **Worst case** = aggregate of peaks = Σ peaks *: 9 + 12 + 6 = 27*
- **Best case** = max(Σ averages, largest single peak) *: 12 · 14*
- **From a table:** sum each interval, and the largest sum is the peak of the aggregate. Ratio = peak of aggregate ÷ Σ peaks *: 70 ÷ 140 = 50%*
- Peak of aggregate ≤ aggregate of peaks

TCP congestion window

- Start at **1 MSS** → **double** each burst, **capped at the threshold** → then **+1 MSS** → stop at the **advertised window** (no window given = no cap)
- **Timeout:** threshold = ⌊½ × current window⌋, window = 1 MSS *: 41 → 20, 3, 6, 12, 20, 23*
- "After the n-th" = the window *after* burst n *: MSS 5, threshold 70: 5, 10, 20, 40, 70, 75, 80 → 85*
- Time = (bursts before the target) × RTT · **"time to send X KB"**: keep a running total. The burst that takes it past X is sent after (n − 1) RTTs *: 4, 8, 16, 32, 50, 54 … → 10th burst → 9 × 15 = 135 ms* · **"how many transmissions"** = n, the burst where the total reaches X *: MSS 3, threshold 81: 3, 6, 12, 24, 48, 81, 84, 87 → total 345 at the 8th*
- **Throughput** = window ÷ RTT (RTT = 2 × one-way) *: 131,072 B ÷ 20 ms = 52 Mb/s*

IPv4 and subnetting

- 32 bits, 2^{32} ≈ 4.3 billion · bit values `128 64 32 16 8 4 2 1`
- **Usable hosts** = $2^{(32-n)}$ − 2 *: /11 → 2,097,150 · /18 → 16,382 · /21 → 2,046*
- **Total addresses** = $2^{(32-n)}$ *: the key's answer for "137.43 network" was 65,536*
- **Subnets** = $2^{(\text{mask n} - \text{starting prefix})}$. Use the /n given in the question, otherwise the class bits (A 8 · B 16 · C 24) *: 141.43.201.17/13 with /21 → 2^8 = 256 · 102.x with /20 → 2^{12} = 4,096*
- **Mask for S subnets:** borrow $\log_2$S bits, so new /n = old /n + $\log_2$S *: 116.143.102.15/13, 512 subnets → 9 bits → /22 = 255.255.252.0*
- **Block** = 256 − mask octet · start = ⌊octet ÷ block⌋ × block
- **Subnet** = copy · start · 0s · **Broadcast** = copy · next start − 1 · 255s
- **Valid host:** host bits not all 0s and not all 1s · an address equal to the broadcast isn't a host, even in the same subnet *(156.1.15.255/20)*
- **Broadcast endings:** cut in the 4th octet → the last octet = a multiple of the block − 1 *(/26: 63, 127, 191, 255 · /28: 15, 31 …, so always odd)* · cut in the 3rd octet (/17–/23) → the broadcast always ends **.255** and the subnet **.0**, so .0 and .255 can be valid hosts · /24 (Class C) → ends in a single .255
- **Hosts per class:** A 16,777,214 · B 65,534 · C 254

Mask octet	Binary	1s	Block	/n in 2nd	/n in 3rd	/n in 4th	Hosts if 4th
128	10000000	1	**128**	/9	/17	/25	126
192	11000000	2	**64**	/10	/18	/26	62
224	11100000	3	**32**	/11	/19	/27	30
240	11110000	4	**16**	/12	/20	/28	14
248	11111000	5	**8**	/13	/21	/29	6
252	11111100	6	**4**	/14	/22	/30	2
254	11111110	7	**2**	/15	/23	/31	0
255	11111111	8	**1**	/16	/24	/32	–

Block = 256 − mask octet = $2^{(8\ -\ 1s)}$. Cut in the 3rd octet: hosts = block × 256 − 2 (e.g. /22: 4 × 256 − 2 = 1,022). /8, /16 and /24 mean the mask octet is 255 or 0, so no block is needed.

Class	A 1–126	B 128–191	C 192–223	D 224–239	E 240+

> e.g. 192.168.153.33/26 → broadcast .63 · /21 → 192.168.159.255

Definitions and true/false facts · from the solution sets and lecture notes

The Internet and layers

- The Internet gives **best effort** service, **no guarantees**
- It is **federated**, with **no central control**, and **asynchronous** (not synchronous)
- Bandwidth varies by **billions** · latency varies by **millions**
- **OSI** has **7 layers** (not 4): physical, data link, network, transport, session, presentation, application
- **Layer 2** = Data Link · **Transport** = the first **end-to-end** layer
- Actual communication in OSI is **vertical**, not horizontal
- **Layering:** a layer **cannot** look inside other layers
- **Encapsulation** hides headers between layers. It does **not** make data secure.
- IP = Internet Protocol · Barbara Liskov won the **Turing Award** (2008), **not** a Nobel Prize

SDU, PCI and PDU

- **SDU** = the data from the layer above · **PCI** = this layer's header · **PDU** = PCI + SDU
- **N+1 PDU = N SDU** · a Transport PDU = a Network SDU · **adding a Network PCI to a Transport PDU gives a Network PDU**
- **Network PDU** = Data Link PDU with the **Data Link PCI** removed
- **Wrong:** adding a Network PCI to a Network PDU gives a Data Link SDU · removing a Network PCI from a Transport PDU · a Transport SDU = a Network PDU

TCP and UDP

- **MSS** = the maximum segment size a device can send or receive, *before* the TCP and IP headers are added. It is set once and **doesn't change**.
- **Congestion window** changes a lot · it **can't exceed** the advertised window
- The **advertised window** can change at any time
- The **threshold can be greater** than the advertised window · the congestion window **can be greater** than the threshold (linear phase)
- IP is connectionless and unreliable, but **TCP** still gives a **reliable, connection-oriented** service over it
- A timeout is assumed to be **congestion** · TCP is self-clocking through ACKs
- TCP is a byte stream, so it **doesn't preserve** message boundaries
- **UDP** is connectionless, with **no connection setup** · it **can deliver packets out of order**

> Answer key: “In UDP an IP host and port uniquely defines a host's application process” is marked **FALSE**

Addressing

- IPv4 = **32 bits** (not 48) · **over 4 billion** addresses
- **ICANN** controls all assignment · **RIPE** = Europe · **APNIC** = Asia Pacific · **ARIN** = Americas
- There are **more Class C networks** than Class B, and **more Class B** than Class A · Class A = **1–126** (“0–127” marked false)
- Hostname → **many IPs** · IP → **many domain names** · domain → **many MACs**
- An IP **doesn't have** many MACs · a MAC **doesn't support** many IPs
- **DNS:** name → IP · **ARP:** IP → MAC · **RARP:** MAC → IP
- A mask must be contiguous 1s then 0s, so **255.255.255.170 is invalid**
- A broadcast doesn't always end in 255 · an octet over 255 (261, 300) is invalid
- Subnetting splits one network into parts. **CIDR** joins prefixes on any bit boundary. They are **not the same**, but you **can** have both at the same time.
- 137.x is Class B (not C) · Class B has **65,534** hosts (not 65,536 or 65,636)

Switching

- **Circuit:** setup, then a reserved link held for the whole connection, then release
- **Packet:** links are **shared** · store-and-forward · pipelining
- There are 2 types of packet switching: **datagram** and **virtual circuit**
- **Datagram:** packets may arrive **out of order** · **better for bursty** traffic than circuit switching
- **Virtual circuit:** in order · links **shared** · still needs **setup and release** · **not** the same as circuit switching
- Neither VC nor datagram, nor packet nor circuit, is **always** better
- Much more bandwidth in circuit switching **doesn't always** cut delay much (setup dominates)

Delays and queueing

- Higher bit rate → lower transmission delay · doubling the rate **doesn't halve** the overall delay
- Doubling the size doubles T_t · doubling the length doesn't change T_t · 10× size with 10× bandwidth gives the same T_t
- Propagation is **not always** bigger than transmission
- A **smaller packet after a larger one** at the same in/out rate **queues** (not in every case)
- Queueing **can happen** even with one input and one output · input much slower than output: “there can be no queueing” is marked **false**
- A **bigger packet after a smaller one**: “there can be no queueing” is marked **false**
- Bigger packet → higher error probability
- One link isn't simply “better”: it depends on packet size (bandwidth vs propagation)
- 2 MB at 2 Mb/s takes 3× as long as 3 MB at 9 Mb/s (8 s vs 2.67 s)

Multiplexing

- **Statistical multiplexing:** share the link rather than provisioning for the worst case
- The aggregate of peaks is usually **much bigger** than the peak of the aggregate
- Many bursty flows together → the peak of the aggregate ≈ the **average**
- The peak of the aggregate **can't be smaller** than the sum of the averages